GPars Integration into Groovy: Three Options Compared

Targeting Groovy 6, JDK 17 minimum, assuming the Groovy async proposal (PR #2437 / GROOVY-9381) is merged.

Option 1: GPars as a Single Groovy Subproject

Bring the GPars repository in as `groovy-gpars` . Update to Groovy 6 / JDK 17. Strip the dead weight (STM/Multiverse, CSP/JCSP, Remoting/Netty). Ship as one module.

What users get

- The full GPars API: parallel collections, actors, dataflow, agents, fork/join
- PGroup as the central integration point stays intact
- Existing GPars documentation, examples, and mental model carry over
- Java users can use GPars (with Groovy on the classpath)

Pros

- Simplest migration — minimal refactoring
- Actor↔dataflow coupling doesn't need to be untangled
- Community familiarity preserved

Cons

- Monolith — users wanting parallel collections pull in actors, dataflow, agents
- Depends on Groovy core — not usable from pure Java without the Groovy runtime
- Overlaps with Groovy async: `AsyncChannel` vs dataflow, `Awaitable` vs `Promise` , `AsyncScope` vs PGroup
- Versioning tied to Groovy release cadence

Option 2: Decomposed into Groovy Modules

Break GPars into focused Groovy subprojects: `groovy-parallel-core` , `groovy-parallel-collections` , `groovy-dataflow` , `groovy-actors` , `groovy-agents` . All depend on Groovy core. Drop CSP, STM, remoting.

What users get

- Pick only what you need — parallel collections without actors, dataflow without agents
- Clean Groovy-idiomatic APIs: `withPool { items.eachParallel { } }`
- Full integration with Groovy async via `AwaitableAdapter`
- AST transforms: `@ActiveObject` , `@AsyncFun` , enhanced `@Parallel`

Pros

- Modular — fine-grained dependency management
- Deep Groovy async integration (`await` dataflow variables, actors, agents)
- Can evolve modules independently
- No external dependencies (Multiverse, JCSP, Netty all dropped)

Cons

- All modules depend on Groovy core — Java users need the Groovy runtime
- Medium refactoring effort (decouple PGroup, move MessageStream, remove setup)
- Groovy-centric API: `Closure` -based, category methods — not idiomatic for Java callers

Option 3: Java-First Modules with Groovy Sugar Layer

Build the core parallelism modules in pure Java using `java.util.function` interfaces. Add a thin Groovy layer on top that provides the idiomatic Groovy experience (categories, `withPool()` , `Closure` -based APIs, AST transforms).

Architecture

```
groovy-parallel-core              (pure Java, →JK files, no Groovy dependency)
  Pool, Scheduler, GParkerFactory, AsyncMessagingCore,
  MessageStream, ForkJoinUtils, concurrency primitives

groovy-parallel-collections   (pure Java, →15 files, no Groovy dependency)
  ParallelCollections static API (java.util.functions),
  AbstractForkJoinWorker, ParCollScope

groovy-dataflow               (pure Java, →JK files, no Groovy dependency)
  DataflowVariable, Promise, DataflowStream,
  Operators, Pipeline — using CompletionStage/Consumer/Function

groovy-actors                 (pure Java, →JK files, no Groovy dependency)
  Actor, DefaultActor, DynamicDispatchActor,
  Mixing (closed!) — message handlers as Consumer<Function

groovy-agents                 (pure Java, →4 files, no Groovy dependency)
  Agent — mutations as Consumer/UnaryOperator

groovy-parallel-groovy        (Groovy, →JK files, depends on all above + Groovy core)
  GPars(Pool, withPool), category methods (eachParallel etc.),
  ParCollEnhancer, TransparentParallel,
  @ActiveObject, @AsyncFun, @Parallel enhancements,
  AwaitableAdapters for dataflow/actors/agents,
  Closure-to-function bridge utilities
```

What Java users see

```
// ParallelCollections (pure Java, no Groovy needed)
var pool = new PJPool(8);
try {
    ParallelCollections.each(pool, items, item -> process(item));
    List<String> results = ParallelCollections.collect(pool, items, Item::getName);
    List<Item> valid = ParallelCollections.findAll(pool, items, Item::isValid);
    Map<String, List<Item>> grouped = ParallelCollections.groupBy(pool, items, Item::getCategory);
} finally {
    pool.shutdown();
}

// Dataflow (pure Java)
var x = new DataflowVariable<Integer>();
var y = new DataflowVariable<Integer>();
pool.execute(() -> x.bind(expensiveComputation()));
pool.execute(() -> y.bind(anotherComputation()));
int sum = x.get() + y.get();  // blocks until both bound

// Actors (pure Java)
var actor = new DynamicDispatchActor<Object> {
    void onMessage(String msg) { replyLong.toUpperCase(); }
};
actor.start();
CompletableFuture<Object> result = actor.sendAndPromise("hello");
```

What Groovy users see (with groovy-parallel-groovy on classpath)

```
// All the familiar Groovy idioms, delegating to the Java API underneath
GParsPool.withPool {
    items.eachParallel { process(it) }
    def valid = items.findAllParallel { it.isOk() }
    def sum2 = items.findAllParallel { it.isOld() }
}

// Dataflow with sugar
def x = new DataflowVariable()
async { x << expensiveComputation() }
def result = await x  // via AwaitableAdapter

// Active objects
@ActiveObject
class MyService {
  @AsyncFunMethod
  def processData( ) /* actor-backed */ }
}
```

Pros

- Java users get real parallelism capabilities without the Groovy runtime
- Usable from Kotlin, Scala, or any JVM language
- Groovy users lose nothing — the sugar layer provides all the familiar idioms
- Modular — same fine-grained dependency story as Option 2
- Testable from Java — core logic doesn't need Groovy test infrastructure
- No external dependencies in any module
- The Groovy sugar layer is thin (~20 files of delegation and categories)

Cons

- Highest refactoring effort — every `Closure` parameter needs a `java.util.function` equivalent
- Two API surfaces to maintain (Java functional + Groovy Closure)
- Some GPars patterns don't map cleanly to Java functional interfaces (multi-arg closures, `curry()` , delegation)
- The Groovy sugar layer needs bridging code (`Closure` → `Consumer` / `Function` adapters)
- PGroup refactoring is more invasive — the facade must work with both Java and Groovy types

Side-by-Side Comparison

Criterion	Option 1: Monolith	Option 2: Groovy Modules	Option 3: Java-First
Modularity	None — all or nothing	Good — pick what you need	Good — pick what you need
Java usability	Needs Groovy runtime; Closure-based API	Needs Groovy runtime; Closure-based API	Pure Java API; no Groovy runtime needed
Groovy experience	Full GPars idioms	Full GPars idioms	Full GPars idioms (via sugar layer)
Kotlin/Scala/other JVM	Awkward (Closure interop)	Awkward (Closure interop)	Natural (Java.util.function)
Async/Awaitable integration	Adapter bolted on	Deep — native Awaitable support	Layered — Java uses CompletableFuture, Groovy layer adds Awaitable
External dependencies	None (after pruning STM/CSP/Remote)	None	None
Migration effort	Low	Medium	High
API surface maintenance	Single (Groovy)	Single (Groovy)	Dual (Java functional + Groovy sugar)
AST transforms	@ActiveObject, @AsyncFun as-is	@ActiveObject, @AsyncFun, enhanced @Parallel	Same as Option 2, but in sugar layer
Overlap with Groovy async	Messy — competing abstractions	Clean — AwaitableAdapter bridges	Cleanest — Java layer uses standard types, Groovy layer bridges to Awaitable

The Async/Awaitable Question Under Option 3

This is where Option 3 actually has an advantage over Option 2, not a disadvantage. The key insight is about which abstraction lives where.

The problem with Awaitable in Java-first modules

`Awaitable` lives in `groovy.concurrent` (core Groovy). If the Java-first modules can't depend on Groovy core, they can't reference `Awaitable` directly. So how do you make `DataflowVariable` or actor replies `await` -able?

The solution: CompletionStage as the universal bridge

Java already has its own "awaitable" type: `java.util.concurrent.CompletableFuture` (which implements `CompletionStage`). This is the standard JDK type for async results. The Groovy async proposal's `AwaitableAdapter` SPI can adapt any type — and `CompletableFuture` is almost certainly already supported out of the box.

The layering:
Java modules return `CompletableFuture<T>` from async operations.
Groovy sugar layer registers `AwaitableAdapter` implementations that bridge GPars types to `Awaitable` .
Result: Java users get `CompletableFuture` (the JDK standard); Groovy users get `await` support. Neither compromises.

How each module adapts

Module	Java API returns	Groovy sugar adds
`groovy-parallel-collections`	`ListOps` (synchronous result, pool handles waiting)	Category methods, `withPool()`
`groovy-dataflow`	`DataflowVariable<T>` implements `CompletionStage<T>`	`AwaitableAdapter` SO `await dfVar` works; `<<` operator; `for await` on channels
`groovy-actors`	`sendAndPromise()` returns `CompletableFuture<T>`	`AwaitableAdapter` ; `@ActiveObject` / `@ActiveMethod` transforms
`groovy-agents`	`getAsync()` returns `CompletableFuture<T>`	`AwaitableAdapter` ; `@Agent` field transform

Concrete example: DataflowVariable

In the pure Java `groovy-dataflow` module:

```
public class DataflowVariable<T> implements CompletionStage<T> {
    public void bind(T value) { ... }
    public T get() { ... } // blocks

    // CompletionStage methods for composition
    public <U> CompletionStage<U> thenApply(Function<T, U> fn) { ... }
    public CompletionStage<Void> thenAccept(Consumer<T> action) { ... }
    public CompletableFuture<T> toCompletableFuture() { ... }
}
```

In the Groovy sugar layer (`groovy-parallel-groovy`):

```
// AwaitableAdapter registered via META-INF/services
class DataflowVariableAwaitableAdapter implements AwaitableAdapter {
    boolean supports(Object obj) { obj instanceof DataflowVariable }
    Awaitable<Object> adapt(Object obj) {
        ((DataflowVariable)obj).toCompletableFuture()
    }
}

// Extension methods for Groovy syntactic sugar
static <T> T leftShift(DataflowVariable<T> self, T value) {
    self.bind(value); return self
}
```

Java user experience:

```
var x = new DataflowVariable<Integer>();
executor.execute(() -> x.bind(42));

// Option A: blocking
int result = x.get();

// Option B: async composition (standard CompletionStage)
x.thenApply(v -> v * 2)
 .thenAccept(System.out::println);
```

Groovy user experience:

```
def x = new DataflowVariable()
async { x << 42 }

// Groovy await Just AwaitableAdapter!
def result = await x

// Or dataflow composition
def doubled = x.then { it * 2 }
```

Why this is actually cleaner

In Option 2, every GPars module would define its own `Promise` type and its own `AwaitableAdapter` . The adapter is tightly coupled to both Groovy's `Awaitable` and GPars' internal type.

In Option 3, the Java modules use `CompletionStage` — the JDK standard that everyone already knows how to compose. The Groovy layer only needs one generic adapter for `CompletionStage` (which the async proposal likely ships with anyway). Each GPars module doesn't need its own adapter — it just implements the standard interface.

Fewer adapters. Standard types. Universal interop.

What about `AsyncChannel` vs `DataflowQueue` ?

The Groovy async proposal introduces `AsyncChannel` (Go-style channels). GPars has `DataflowQueue` (similar concept, but participates in operator networks).

Under Option 3, `DataflowQueue` is a pure Java type that implements `ITerableT>` and/or a `BlockingQueue<T>` -like interface. The Groovy sugar layer could make it work with `for await` via an adapter. Meanwhile, `AsyncChannel` remains the lightweight Go-style primitive in core Groovy. They coexist at different abstraction levels:

- `AsyncChannel` — simple point-to-point, Go-style, built into the language
- `DataflowQueue` — participates in operator networks, pipelines, select/guard; richer semantics

Final Assessment

If your priority is…	Best option
Ship something fast, minimal effort	**Option 1** — bring it in, prune the dead weight, ship
Best Groovy experience, deep async integration	**Option 2** — Groovy-native modules with Awaitable woven in
Broadest audience (Java + Groovy + other JVM), cleanest architecture	**Option 3** — Java-first with Groovy sugar layer
Future-proofing (virtual threads, Project Loom, evolving JDK)	**Option 3** — pure Java core evolves with the JDK naturally

Recommendation: Option 3 is the most work upfront but yields the cleanest result. The async/Awaitable story is actually *simpler* under Option 3 because you lean on `CompletionStage` as the universal bridge rather than building custom adapters for each module. And the Groovy experience doesn't suffer — the sugar layer is thin delegation that provides all the familiar idioms.

A pragmatic middle ground: start with Option 2 for the initial release (Groovy modules, fast to ship), but design the internal APIs with `java.util.function` interfaces from day one. This makes a future migration to Option 3 straightforward — you'd be extracting the already-clean Java internals into their own modules and leaving the Groovy wrappers behind.